CMSE 801 - Day 12

Using Pandas to manipulate, clean, and explore data

October 7, 2024

General Course Announcements

- Still working on grading the quiz -- sorry for the delay on getting that back. Aiming for end of this week so that you have the result before Quiz 2.
    - If you have any questions about the quiz or want to discuss problems you ran into, please contact me or come to office hours.
    - Partial credit will be given where possible!
- Homework 2 is **available** and is due **this Friday**, October 11, by 11:59pm.

Questions/comments from Day 12 PCA

- Getting used to Pandas:
    - Questions about the difference between using `.loc` (label-based) and `.iloc` (index-based) and how to use each (especially using `,` and `:`)
    - Questions about accessing the row labels and column labels

Loading the modules and then the heart disease example data

In [1]:

```
# import numpy
import numpy as np

# import matplotlib
import matplotlib.pyplot as plt

# Look at the Pandas import -- we take a similar approach to how we impo
# "pd" will be the short-hand for accessing Pandas functions.
import pandas as pd

# Because we're looking data stored directly in this notebook, we need t
from io import StringIO

# read in some data on heart disease (don't worry too much about what "j
heart_disease_data = pd.read_json(StringIO("""[{"observation_number":0,"
```

Let's explore the data bit

In [2]:
```
heart_disease_data.head()
```

Out[2]:

	observation_number	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exa
0	0	67	1	4	160	286	0	2	108	
1	1	67	1	4	120	229	0	2	129	
2	2	37	1	3	130	250	0	0	187	
3	3	41	0	2	130	204	0	2	172	
4	4	56	1	2	120	236	0	0	178	

In [3]:
```
heart_disease_data.describe()
```

Out[3]:

	observation_number	age	sex	cp	trestbps
count	302.000000	302.000000	302.000000	302.000000	302.000000
mean	150.500000	54.410596	0.678808	3.165563	131.645695
std	87.324109	9.040163	0.467709	0.953612	17.612202
min	0.000000	29.000000	0.000000	1.000000	94.000000
25%	75.250000	48.000000	0.000000	3.000000	120.000000
50%	150.500000	55.500000	1.000000	3.000000	130.000000
75%	225.750000	61.000000	1.000000	4.000000	140.000000
max	301.000000	77.000000	1.000000	4.000000	200.000000

What if I just want to access some columns of data?

In [7]:
```
heart_disease_data['age']
```

Out[7]:
```
0      67
1      67
2      37
3      41
4      56
      ..
297    45
298    68
299    57
300    57
301    38
Name: age, Length: 302, dtype: int64
```

In [5]:
```
heart_disease_data['trestbps']
```

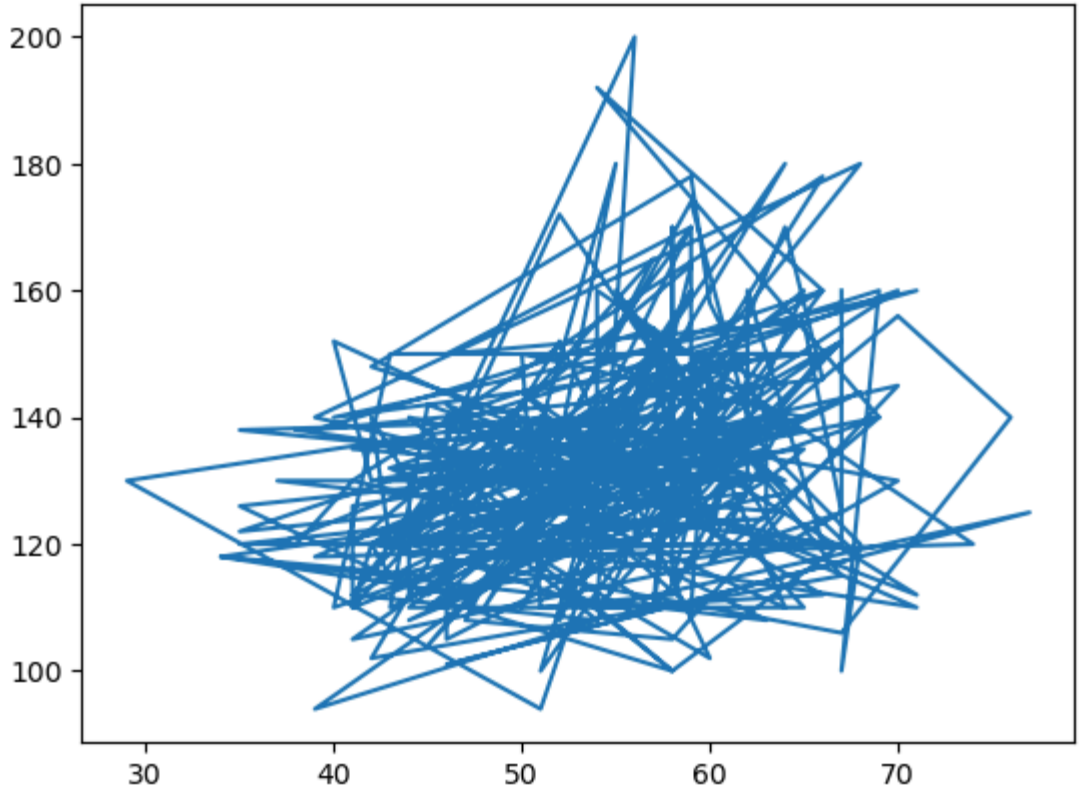
Out[5]:

```
0      160
1      120
2      130
3      130
4      120
      ...
297    110
298    144
299    130
300    130
301    138
Name: trestbps, Length: 302, dtype: int64
```

I can use this sort of data access to quickly make plots of one column against another

In [8]:
```
plt.plot(heart_disease_data['age'],heart_disease_data['trestbps'])
```

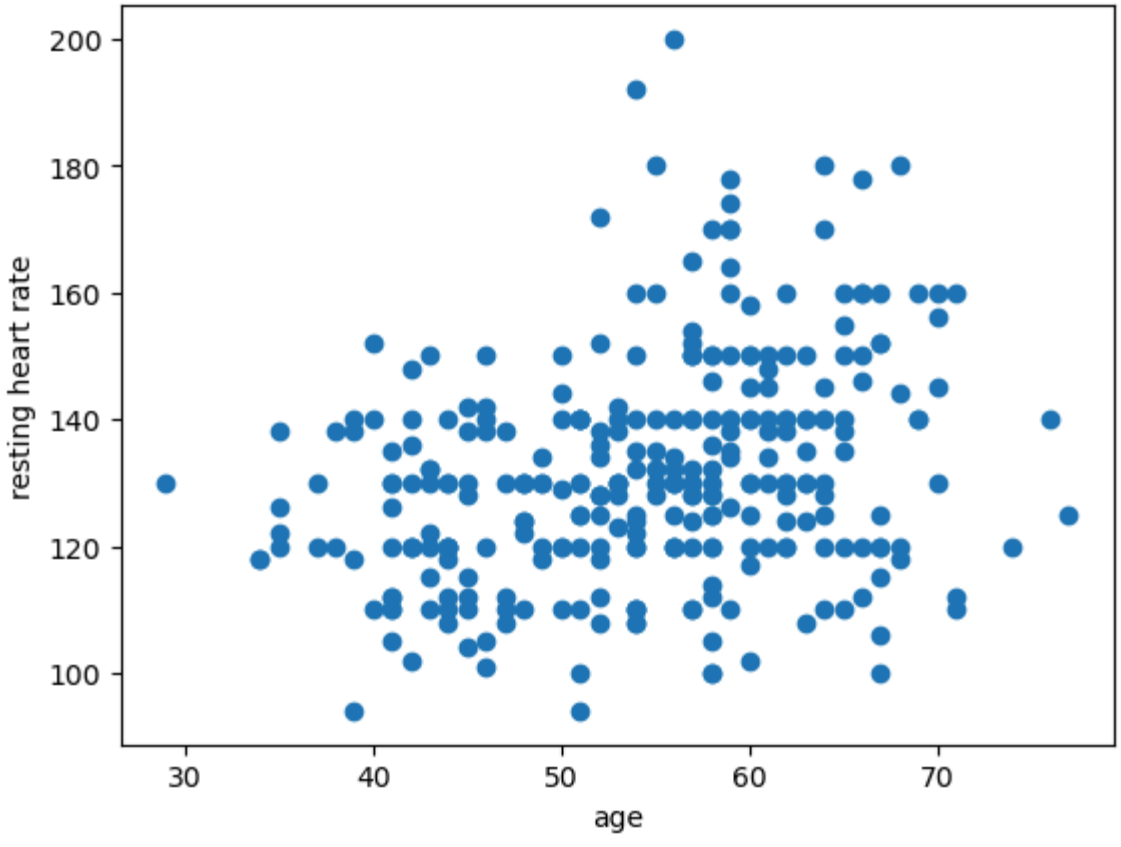
Out[8]:
```
[<matplotlib.lines.Line2D at 0x13051e0c0>]
```



I can do better!

In [11]:
```
plt.scatter(heart_disease_data['age'],heart_disease_data['trestbps'])
#plt.plot(heart_disease_data['age'],heart_disease_data['trestbps'],ls="'
plt.xlabel("age")
plt.ylabel("resting heart rate")
```

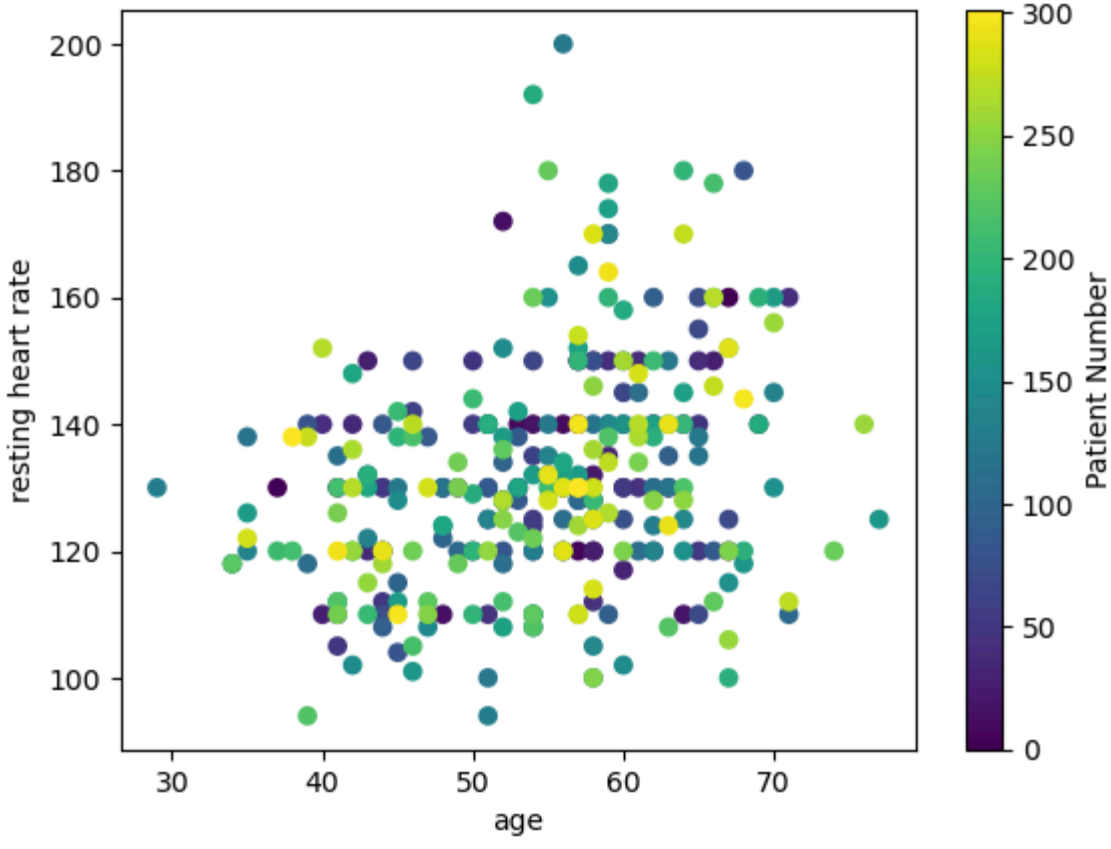
Out[11]:
```
Text(0, 0.5, 'resting heart rate')
```


200
180
160
140
120
100
resting heart rate
30
40
50
60
70
age

What does each dot represent on this plot?

Each dot is the "age" and "trestbps" for a single observation in my dataset. I can even visualize this, if I want!

In [12]:
```
plt.scatter(heart_disease_data['age'],heart_disease_data['trestbps'], c=
plt.xlabel("age")
plt.ylabel("resting heart rate")
cb = plt.colorbar()
cb.set_label("Patient Number")
```


resting heart rate
200
180
160
140
120
100
30
40
50
60
70
age
Patient Number
300
250
200
150
100
50
0

Accessing the data in other ways (e.g. using `.loc` and `.iloc`)

Important note: when using `.loc` and `.iloc`, the first value you provide is to access **row** information, the second value is to access **column** information.

In [13]:
```
heart_disease_data.head()
```

Out[13]:

	observation_number	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exa
0	0	67	1	4	160	286	0	2	108	
1	1	67	1	4	120	229	0	2	129	
2	2	37	1	3	130	250	0	0	187	
3	3	41	0	2	130	204	0	2	172	
4	4	56	1	2	120	236	0	0	178	

In [14]:
```
heart_disease_data.loc[3] # This is "label-based" access. I'm accessing
```

Out[14]:

```
observation_number      3
age                    41
sex                     0
cp                      2
trestbps              130
chol                  204
fbs                     0
restecg                 2
thalach               172
exang                   0
oldpeak               1.4
slope                   1
ca                    0.0
thal                  3.0
num                     0
Name: 3, dtype: object
```

In [15]:
```
heart_disease_data.head()
```

Out[15]:

	observation_number	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exa
0	0	67	1	4	160	286	0	2	108	
1	1	67	1	4	120	229	0	2	129	
2	2	37	1	3	130	250	0	0	187	
3	3	41	0	2	130	204	0	2	172	
4	4	56	1	2	120	236	0	0	178	

In [16]:
```
heart_disease_data.loc[3,'age'] # This is "label-based" access. I'm acce
                                # and then the column labeled "age"
```

Out[16]:
```
41
```

In [17]:
```
heart_disease_data.head()
```

Out[17]:

	observation_number	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exa
0	0	67	1	4	160	286	0	2	108	
1	1	67	1	4	120	229	0	2	129	
2	2	37	1	3	130	250	0	0	187	
3	3	41	0	2	130	204	0	2	172	
4	4	56	1	2	120	236	0	0	178	

In [18]:
```
heart_disease_data.iloc[2]  # This is "index-based" access. I'm accessi
```

Out[18]:

```
observation_number      2
age                    37
sex                     1
cp                      3
trestbps              130
chol                  250
fbs                     0
restecg                 0
thalach               187
exang                   0
oldpeak               3.5
slope                   3
ca                    0.0
thal                  3.0
num                     0
Name: 2, dtype: object
```

In [19]:
```
heart_disease_data.head()
```

Out[19]:

	observation_number	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exa
0	0	67	1	4	160	286	0	2	108	
1	1	67	1	4	120	229	0	2	129	
2	2	37	1	3	130	250	0	0	187	
3	3	41	0	2	130	204	0	2	172	
4	4	56	1	2	120	236	0	0	178	

In [20]:
```
heart_disease_data.iloc[2,4]  # This is "index-based" access. I'm acces
                              # and then the column at index 4
```

Out[20]:
```
130
```

I can also get tricky and use the ":" to access a larger subset of the data

In [21]:
```
heart_disease_data.head()
```

Out[21]:

	observation_number	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exa
0	0	67	1	4	160	286	0	2	108	
1	1	67	1	4	120	229	0	2	129	
2	2	37	1	3	130	250	0	0	187	
3	3	41	0	2	130	204	0	2	172	
4	4	56	1	2	120	236	0	0	178	

In [22]:
```
heart_disease_data.iloc[2:4,4:9]
```

Out[22]:

	trestbps	chol	fbs	restecg	thalach
2	130	250	0	0	187
3	130	204	0	2	172

In [23]:
```
heart_disease_data.loc[2:4,"age":"trestbps"]
```

Out[23]:

	age	sex	cp	trestbps
2	37	1	3	130
3	41	0	2	130
4	56	1	2	120

How do I get access to the row labels and columns labels?

In [24]:
```
heart_disease_data.index
```

Out[24]:
```
RangeIndex(start=0, stop=302, step=1)
```

In [25]:
```
heart_disease_data.index.values
```

Out[25]:

```
array([  0,   1,   2,   3,   4,   5,   6,   7,   8,   9,  10,
11,  12,
        13,  14,  15,  16,  17,  18,  19,  20,  21,  22,  23,
24,  25,
        26,  27,  28,  29,  30,  31,  32,  33,  34,  35,  36,
37,  38,
        39,  40,  41,  42,  43,  44,  45,  46,  47,  48,  49,
50,  51,
        52,  53,  54,  55,  56,  57,  58,  59,  60,  61,  62,
63,  64,
        65,  66,  67,  68,  69,  70,  71,  72,  73,  74,  75,
76,  77,
        78,  79,  80,  81,  82,  83,  84,  85,  86,  87,  88,
89,  90,
        91,  92,  93,  94,  95,  96,  97,  98,  99, 100, 101, 1
02, 103,
       104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 1
15, 116,
       117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 1
28, 129,
       130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 1
41, 142,
       143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 1
54, 155,
       156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 1
67, 168,
       169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 1
80, 181,
       182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 1
93, 194,
       195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 2
```

```
06, 207,
       208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 2
19, 220,
       221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 2
32, 233,
       234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 2
45, 246,
       247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 2
58, 259,
       260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 2
71, 272,
       273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 2
84, 285,
       286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 2
97, 298,
       299, 300, 301])
```

In [28]:
```
heart_disease_data.columns
```

Out[28]:
```
Index(['observation_number', 'age', 'sex', 'cp', 'trestbps', 'c
hol', 'fbs',
       'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca',
'thal', 'num'],
      dtype='object')
```

Goals for today

- Load and explore some data using **Pandas**. (specifically, global GDP data)
- Manipulate that data to clean it up and re-organize it to make it easier to analyze
- Visualize that data using matplotlib or some of the built-in Pandas functions

Looking forward - Day 13 PCA: Reviewing normal distributions, how to ”mask” data in Python, and making logarithmic plots

- You'll explore what is meant when people talk about data having a "normal" distribution
- You'll practice "masking" data using Python
- You'lll practice making a logarithmic plot, which can be better for visualizing data with a large range of values.